Definition: IR Encoder for Universal Graph-Token DSL

IR Encoder for Universal Graph-Token DSL

This page defines the **canonical IR encoder** that converts the Lean DSL graph structures into the reversible graph-token format used by the universal math grammar. The encoder is deterministic, canonical, and fully aligned with the balanced-ternary polarity scheme.
This is **Layer 2A** of the architecture: the Lean → IR direction.

---

1. Polarity Rendering

```
def polarityToString : Polarity → String
 | Polarity.pos => "+"
 | Polarity.zero => "0"
 | Polarity.neg => "-"
```

---

2. Entity Identifier Rendering

```
def entityIdToString : EntityId → String
 | EntityId.var s => s
 | EntityId.num n => "N" ++ toString n
 | EntityId.term k => "T" ++ toString k
 | EntityId.eqn => "EQ"
 | EntityId.fn k => "F" ++ toString k
 | EntityId.mat k => "M" ++ toString k
 | EntityId.vec k => "V" ++ toString k
 | EntityId.set k => "S" ++ toString k
 | EntityId.rv k => "RV" ++ toString k
 | EntityId.op k => "Op" ++ toString k
 | EntityId.obj k => "Obj" ++ toString k
 | EntityId.mor k => "Mor" ++ toString k
 | EntityId.custom s => s
```

---

3. Relation Operator Rendering

```
def relationOpToString : RelationOp → String
 | RelationOp.mul => "mul"
 | RelationOp.add => "add"
 | RelationOp.sub => "sub"
 | RelationOp.div => "div"
 | RelationOp.eq => "eq"
 | RelationOp.diff => "diff"
 | RelationOp.integrate => "integrate"
 | RelationOp.limit => "limit"
 | RelationOp.eval_at => "eval_at"
 | RelationOp.matmul => "matmul"
 | RelationOp.dot => "dot"
 | RelationOp.cross => "cross"
 | RelationOp.transpose => "transpose"
 | RelationOp.inv => "inv"
 | RelationOp.det => "det"
 | RelationOp.union => "union"
 | RelationOp.intersect => "intersect"
 | RelationOp.set_sub => "set_sub"
 | RelationOp.subset => "subset"
 | RelationOp.and => "and"
 | RelationOp.or => "or"
 | RelationOp.not => "not"
 | RelationOp.implies => "implies"
 | RelationOp.iff => "iff"
 | RelationOp.prob => "prob"
 | RelationOp.expect => "expect"
 | RelationOp.var => "var"
 | RelationOp.dist => "dist"
 | RelationOp.compose => "compose"
 | RelationOp.id => "id"
 | RelationOp.assoc => "assoc"
 | RelationOp.custom s => s
```

---

4. Operation Operator Rendering

```
def operationOpToString : OperationOp → String
 | OperationOp.add => "add"
 | OperationOp.sub => "sub"
 | OperationOp.div => "div"
 | OperationOp.iso => "iso"
 | OperationOp.apply_diff_rule => "apply_diff_rule"
 | OperationOp.apply_int_rule => "apply_int_rule"
 | OperationOp.change_var => "change_var"
 | OperationOp.limit_simplify => "limit_simplify"
 | OperationOp.row_reduce => "row_reduce"
 | OperationOp.diag => "diag"
 | OperationOp.eig_decomp => "eig_decomp"
 | OperationOp.simplify_logic => "simplify_logic"
 | OperationOp.simplify_set => "simplify_set"
 | OperationOp.normalize_formula => "normalize_formula"
 | OperationOp.custom s => s
```

---

5. Entity and Attribute Token Encoding

```
def encodeEntity (e : Entity) : String :=
  let p := polarityToString e.polarity
  let id := entityIdToString e.id
  s!"E{p}:{id}{e.chain}"


def encodeAttribute (a : Attribute) : String :=
  let p := polarityToString a.polarity
  let id := entityIdToString a.target
  s!"A{p}:{id}{a.chain}:{a.key}:{a.value}"
```

---

6. Entity Reference Encoding

```
def encodeEntityRef (e : Entity) : String :=
  let p := polarityToString e.polarity
  let id := entityIdToString e.id
  s!"E{p}:{id}{e.chain}"
```

---

7. Relation and Operation Token Encoding

```
def encodeRelation (r : Relation) : String :=
  let p := polarityToString r.polarity
  let op := relationOpToString r.op
  let e1 := encodeEntityRef r.e1
  let e2 := encodeEntityRef r.e2
  let e3 := encodeEntityRef r.e3
  s!"R{p}:{op}{r.chain}:{e1}:{e2}:{e3}"


def encodeOperation (o : Operation) : String :=
  let p := polarityToString o.polarity
  let op := operationOpToString o.op
  let e1 := encodeEntityRef o.e1
  let e2 := encodeEntityRef o.e2
  s!"O{p}:{op}{o.chain}:{e1}:{e2}"
```

---

8. Full Graph Encoder

```
def encodeGraph (g : Graph) : List String :=
  let es := g.entities.map encodeEntity
  let as := g.attributes.map encodeAttribute
  let rs := g.relations.map encodeRelation
  let os := g.operations.map encodeOperation
  es ++ as ++ rs ++ os
```

Purpose of This Page

This encoder provides a **canonical, reversible mapping** from Lean DSL structures to graph-token IR strings. It is the foundation for:

- SLM training
- IR decoding
- Lean autocompletion
- Verified rewrite pipelines

This page is the authoritative reference for the IR encoding layer.